

Software Architecture and Design Specification - Team 11

Project: Snake Game (Console-based)

Version: 1.0

Authors: S Prithika, S N Niharika, S V Bhavana, S Ashwin

SRN: PES2UG24CS415, PES2UG24CS414, PES2UG24CS417, PES2UG24CS413

Date: 03-10-2026

Status: Final

Revision History

Version 1.0 | 03-10-2026 | Team | Initial SAD

Approvals

Role | Name | Signature/Date

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Snake Game, console-based implementation. It translates the functional and non-functional requirements defined in the SRS, into a concrete system structure, component design, interaction flow, and technology plan that developers can implement directly.

1.2 Scope

Covers the architecture and design of the console-based Snake Game: menu and initialization, movement and control, food and growth, collision and game over, scoring, pause and difficulty selection, and the restricted Administrator function for protected game data and stored score and configuration information. The graphics/SFML presentation mode described as an alternative in the SRS is out of scope for this document, which addresses the console build only.

1.3 Audience

Developers, QA/testers, course instructors, and future maintainers of the Snake Game codebase.

1.4 Definitions

- Snake: the player-controlled object that moves around the game board and grows when it consumes food.

- Food: an item placed on an unoccupied grid cell that the snake can consume to increase its length and score.
- Game Board: the fixed-size grid area in which the snake moves and food is placed, rendered as a text grid in the console.
- Game Tick: a single update cycle during which the game state and snake position are updated.
- HUD: the on-screen status area showing the player's score and game status, rendered as console text.
- FR / NFR: Functional Requirement / Non-Functional Requirement, as defined in the SRS.
- ADR: Architecture Decision Record, used to document a chosen design option and its rationale.
- RBAC: Role-Based Access Control, used here to separate Player and Administrator privileges.
- STRIDE: a threat modeling framework covering Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, and Elevation of privilege.
- RTM: Requirements Traceability Matrix, mapping requirements to design elements and test cases.

2. Document Overview

2.1 How to use this document

This document provides the architectural deliverables for the Snake Game: component structure, chosen architecture pattern, technology stack, risk register, requirements traceability, a STRIDE-based security review, sequence diagrams for key flows, internal interface design, and the user experience approach for the console presentation.

2.2 Related Documents

Software Requirements Specification (SRS), Snake Game, Team 11, Version 1.0.

Software Test Plan (STP), to be developed from the acceptance tests in SRS Section 6.

Requirements Traceability Matrix (RTM), SRS Section 8, extended in Section 3.8 of this document to the architecture level.

3. Architecture

3.1 Goals & Constraints

Goals: responsive, stable single-player gameplay with a consistent tick rate; clear separation between game logic and console presentation so the presentation layer can be replaced later without rewriting game rules; restricted and auditable access to the Administrator functions.

Constraints: implementation in C or C++; console-only input and output for this build, using non-blocking keyboard polling; single-player, offline operation with no network dependency; the game loop timing defined in the SRS must be preserved; behavior must remain consistent with the functional and non-functional requirements in the SRS.

3.2 Stakeholders & Concerns

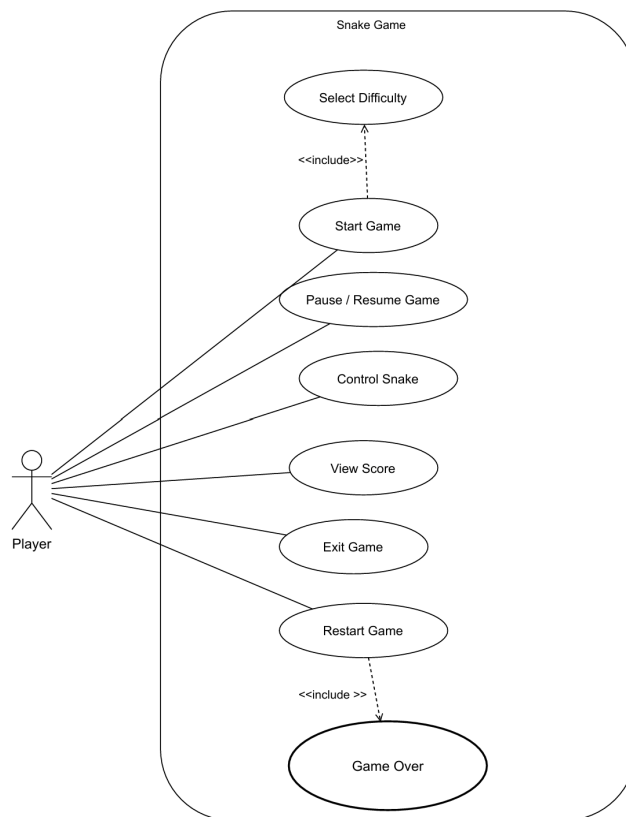
Player: responsive controls, clear and readable console output, stable gameplay with no crashes.

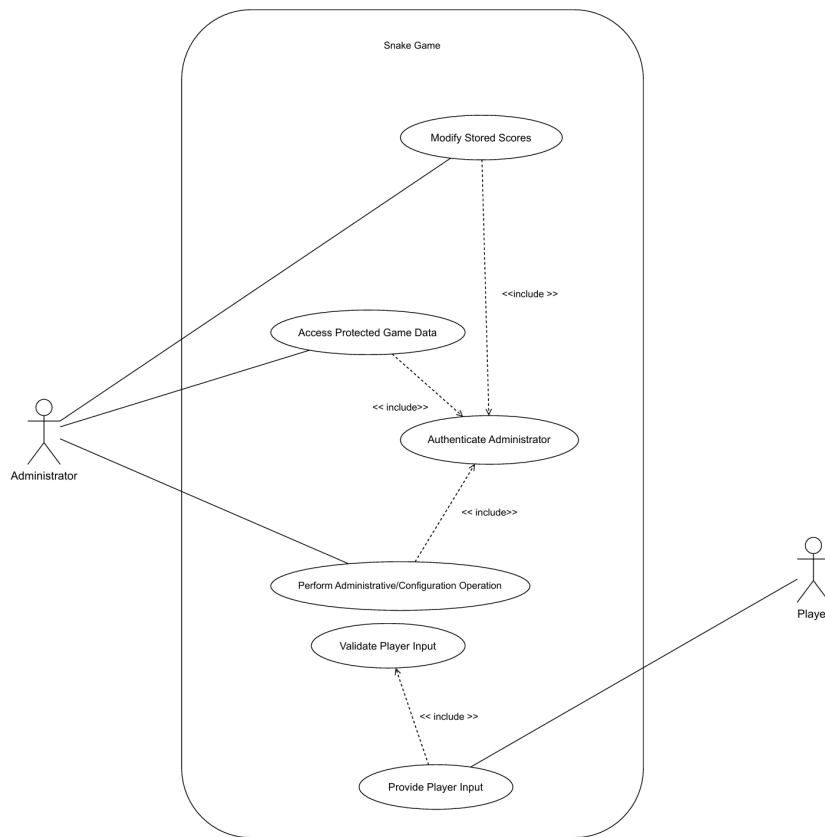
Administrator: restricted, authenticated access to protected game data and stored score and configuration information.

Developers: modular components that can be tested and modified independently, and a presentation layer that can later be swapped for a graphical front end without changing game logic.

Course Instructor: traceability between requirements, design, and test cases, and adherence to the SRS.

3.3 Component (UML) Diagram





3.4 Component Descriptions

- MenuModule: presentation-layer component that renders the Main Menu, handles difficulty selection, and manages navigation between menu, gameplay, pause, and game-over screens.
- RenderModule / Console UI: presentation-layer component that redraws the game board, snake, food, HUD, and Game Over screen to the console on every tick.
- InputModule: control-layer component that polls the keyboard for arrow key or WASD directional input, confirm and pause keys, and forwards validated intent to GameCore.
- GameCore: core component that owns game state, including snake position, food position, score, collision detection, and the overall game loop.
- PersistenceModule: component responsible for reading and writing stored score and configuration data to a local file.
- AdminModule: control-layer component that authenticates the Administrator and exposes access to protected game data and administrative/configuration operations once authenticated.
- ValidationModule: control-layer component that validates player input and configuration values before they reach GameCore or PersistenceModule.

3.5 Chosen Architecture Pattern and Rationale

A layered architecture is chosen, with four layers: a Presentation layer (MenuModule, RenderModule), a Control layer (InputModule, ValidationModule, AdminModule), a

Core/Domain layer (GameCore), and a Persistence layer (PersistenceModule). Within the Core layer, the game loop follows the standard real-time game loop pattern of process input, update state, render, repeated once per tick, and gameplay states such as Menu, Playing, Paused, and Game Over are modeled with a State pattern so each state's behavior and allowed transitions are isolated.

This pattern is chosen because the application is a single-process, single-player console program with no concurrent remote clients, so the added complexity of a client-server or microservices split is not justified. A layered design still gives the separation of concerns the team needs: GameCore can be unit tested without any console I/O, and the Console UI could later be replaced by a graphical front end such as SFML without changing GameCore or AdminModule. Client-server and microservices architectures were considered and rejected as unnecessary overhead for a local, offline, single-user application.

3.6 Technology Stack & Data Stores

- Language: C++17, compiled with g++ or an equivalent C/C++ compiler.
- Console I/O: a platform abstraction over conio.h on Windows and ncurses on Linux, exposing a single non-blocking input and cursor-positioning interface to InputModule and RenderModule so the rest of the codebase stays platform independent.
- Data structures: standard C++ STL containers, for example a deque to represent the snake's body segments for efficient head and tail updates.
- Data store: a local flat file, for example highscores.dat, used by PersistenceModule to store high scores and configuration values. No database server or network storage is used, consistent with the offline, single-player scope.
- Build tooling: a Makefile for compilation, and Git for version control.
- Encryption/TLS: not applicable, since the application does not communicate over a network.

3.7 Risks & Mitigations

- Risk: blocking console input causes missed ticks or unresponsive controls.
Mitigation: InputModule uses non-blocking polling, for example kbhit/getch on Windows or getch with a timeout under ncurses, so the game loop never stalls waiting on a keypress.
- Risk: full-screen console redraws cause visible flicker. Mitigation: RenderModule repositions the cursor and rewrites only changed cells where practical, instead of clearing and redrawing the entire screen every tick.
- Risk: divergent behavior between the Windows and Linux console libraries.
Mitigation: all platform-specific calls are isolated behind the console I/O abstraction, so platform differences are contained in one place.

- Risk: unauthorized access to protected game data or stored scores. Mitigation: AdminModule requires successful authentication before any call reaches PersistenceModule for protected data or configuration changes.
- Risk: a missing or malformed high-score file disrupts startup. Mitigation: PersistenceModule validates the file on load and falls back to a safe, empty default if the file is missing or cannot be parsed.

3.8 Traceability to Requirements

Req ID	Requirement (short)	Component(s)
SNK-F-001, SNK-F-002, SNK-F-006	Main menu, difficulty selection, return/exit	MenuModule
SNK-F-003	Board and snake initialization	GameCore
SNK-F-004	Directional input handling	InputModule
SNK-F-005	Redraw snake and grid each tick	RenderModule
SNK-F-007, SNK-F-008	Score increase, final score display	RenderModule
SNK-F-009, SNK-F-010	Score increase, final score display	GameCore, RenderModule
SNK-F-011, SNK-F-012	Score increase, final score display	GameCore, RenderModule
SNK-F-013, SNK-F-014	Pause/resume, difficulty selection	GameCore, MenuModule
SNK-F-015 to SNK-F-018	Administrator authentication and administrative operations	AdminModule
SNK-NF-001, SNK-NF-002	Input latency, keyboard navigability	InputModule, MenuModule
SNK-NF-003, SNK-NF-004	Stable tick rate, 30 minute reliability	GameCore
SNK-NF-005, SNK-NF-006	Input-to-movement response, distinguishable controls	InputModule, RenderModule
SNK-SR-001	Validate player input and configuration values	ValidationModule
SNK-SR-001	Prevent unauthorized score modification	PersistenceModule
SNK-SR-003	Restrict admin/config operations to authorized users	AdminModule
SNK-SR-004	Protect stored data from unauthorized access or disclosure	PersistenceModule, AdminModule

3.9 Security Architecture

Threat modeling using STRIDE:

- Spoofing: an unauthenticated user claiming to be the Administrator is mitigated by requiring successful authentication in AdminModule before any protected operation runs, per SNK-F-015.
- Tampering: unauthorized modification of player input, configuration values, or stored scores is mitigated by ValidationModule's input checks, per SNK-SR-001, and by PersistenceModule rejecting unauthorized writes, per SNK-SR-002.
- Repudiation: not formally addressed in this release, since the system is a single-user, offline, local application with no multi-user audit requirement in the SRS. This is accepted as an open item rather than mitigated.
- Information disclosure: unauthorized viewing of protected game data or stored configuration is mitigated by gating all such access behind AdminModule authentication, per SNK-SR-004.
- Denial of service: not applicable in the traditional network sense, since the application is offline and single-player. The closest equivalent risk, an unresponsive game loop, is addressed under Reliability in Section 3.7 and by SNK-NF-003 and SNK-NF-004.
- Elevation of privilege: a Player attempting to reach Administrator-only functions is mitigated by role separation enforced in AdminModule, so administrative and configuration operations are only reachable after authentication, per SNK-SR-003.

4. Design

4.1 Design Overview

The Snake Game is designed using four cooperating layers: Presentation, Control, Core/Domain, and Persistence. The Presentation layer consists of MenuModule and RenderModule and is responsible for displaying menus, the game board, snake, food, score, and game status on the console. The Control layer consists of InputModule, ValidationModule, and AdminModule and handles user input, validation, authentication, and administrative access.

The Core layer consists of GameCore, which maintains the authoritative game state and executes the game loop. During each game tick, the system processes user input, updates the game state, checks collisions and food consumption, updates the score, and requests the updated screen to be rendered. The Persistence layer consists of PersistenceModule and is responsible for storing and retrieving high scores and configuration information from a local file.

This separation of responsibilities makes the system modular and easier to test. It also allows the console presentation to be replaced by a graphical interface in the future without changing the main game logic.

4.2 UML Sequence Diagrams

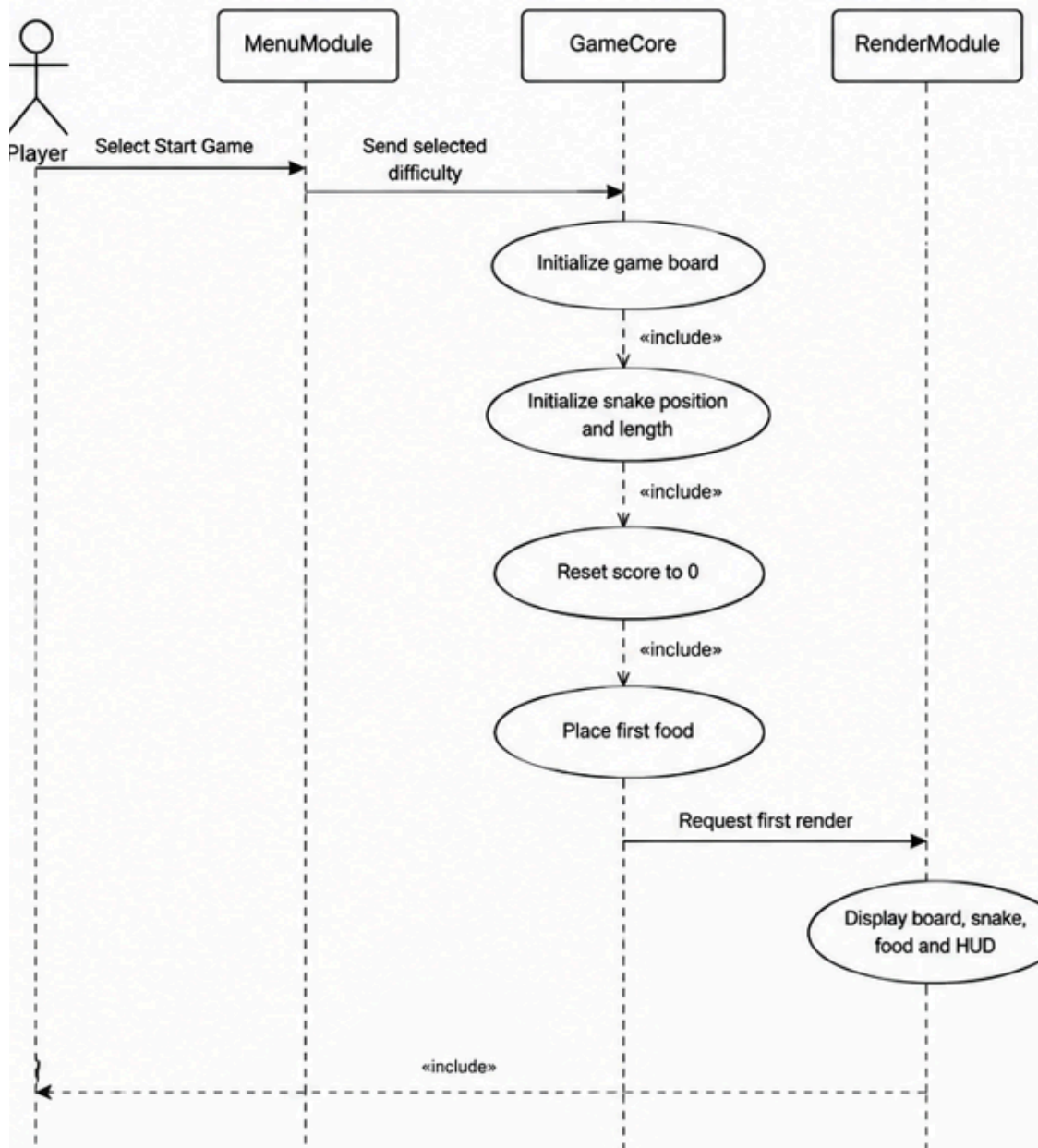
Flow 1: Start Game and First Render

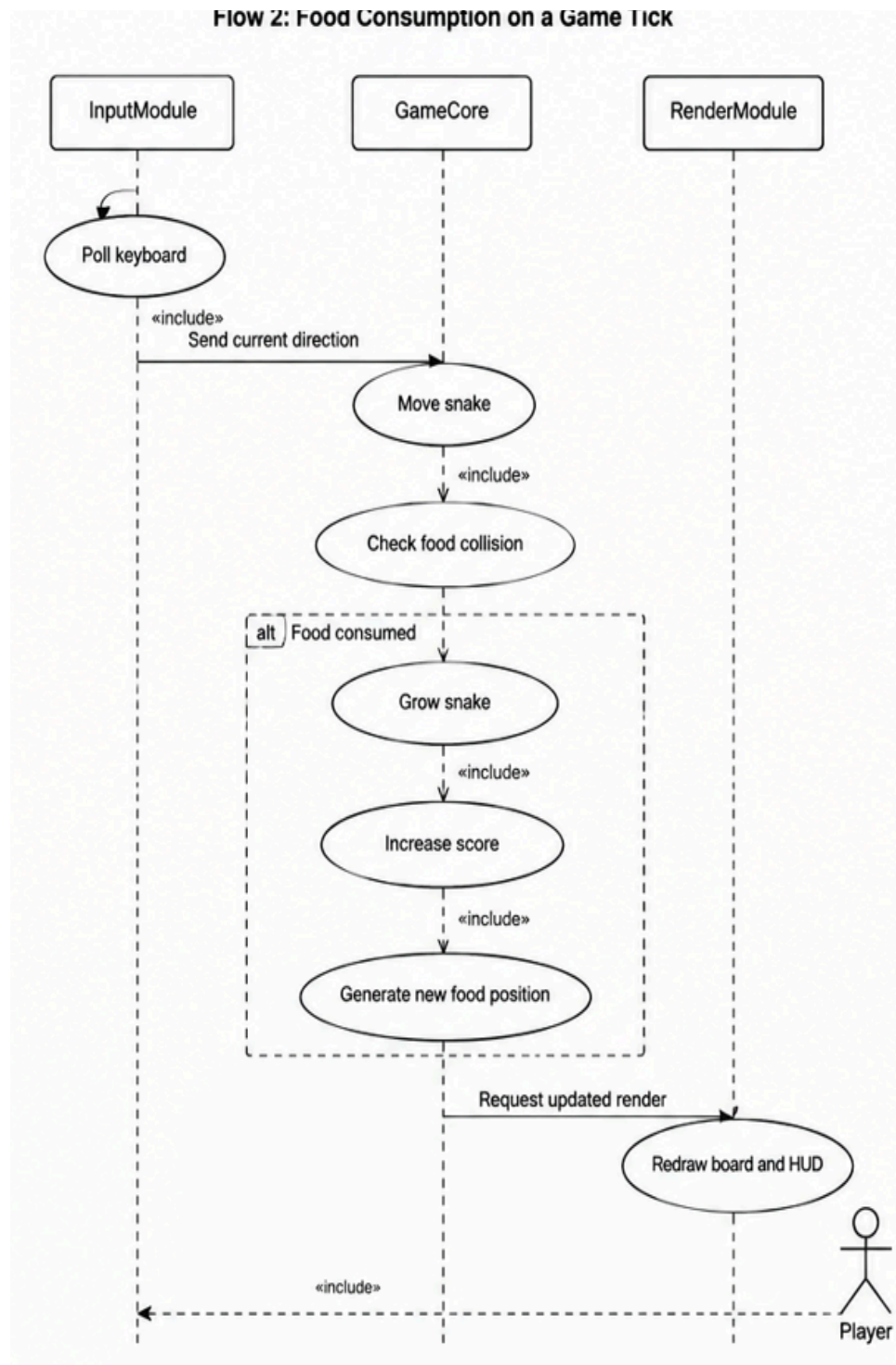
The Player selects Start Game from the Main Menu. MenuModule sends the selected difficulty to GameCore. GameCore initializes the game board, places the snake at its default position, resets the score to zero, and places the first food item. GameCore then requests RenderModule to display the board, snake, food, and HUD.

Flow 2: Food Consumption on a Game Tick

InputModule polls the keyboard and sends the current direction to GameCore. GameCore moves the snake and checks whether the snake's head reaches the food. When food is consumed, GameCore increases the snake's length, increments the score, and generates a new food position. GameCore then requests RenderModule to redraw the updated board, snake, food, and HUD.

Flow 1: Start Game and First Render





4.3 API Design

The console application does not expose a network API. Instead, communication between the system components is handled through internal interfaces. The main interfaces are described below.

Component	Function	Parameters	Returns	Errors
InputModule	pollDirection()	None	Direction (Up, Down, Left, Right, None)	Returns None when no key is pressed; unmapped keys are ignored
GameCore	update(Direction input)	Direction	TickResult containing snake position, food position, score and collision flag	Previous direction is retained if an invalid reversal is requested
AdminModule	authenticate(string credentials)	Credentials	Boolean authentication result	Returns false for invalid credentials without revealing which credential is incorrect
PersistenceModule	saveScore(int score)	Score	Boolean write status	Returns false if validation fails or the file cannot be written

4.4 Error Handling, Logging & Monitoring

Errors are represented internally with a small result/status type rather than exceptions crossing layer boundaries, so GameCore, AdminModule, and PersistenceModule each return a success flag plus a reason code that RenderModule translates into a console message. Administrator credentials and the contents of protected data are never written to any debug output or log file. During development, a build-time debug flag enables verbose console logging of tick timing and state transitions; this flag is disabled in the release build. Since the application is offline and single-user, monitoring is limited to local developer-facing instrumentation, specifically tick duration and input-to-response latency, used to verify

SNK-NF-001, SNK-NF-003, and SNK-NF-005 during testing, and a simple reliability check for crash-free runtime against SNK-NF-004.

4.5 UX Design

The console UI uses a fixed-width text layout for the board, snake, and food, with the HUD showing the current score and, on higher difficulties, the active speed level. Menu options are always keyboard-navigable, with the currently highlighted option shown using a distinguishable marker rather than color alone, since many terminals have limited or no color support. Controls are kept consistent across all screens: arrow keys or WASD for direction, Enter or Space to confirm, and Esc to pause, return, or exit, matching SRS Section 3.2. The Game Over screen clearly states the final score and the two available next actions, Restart or Exit, with the default action highlighted for players who prefer pressing Enter without reading every option.

4.6 Open Issues & Next Steps

- The SRS defines Security Objectives for protecting stored scores and configuration data, but does not include an explicit functional requirement establishing that scores are persisted. This design assumes persistence occurs through PersistenceModule; this assumption should be confirmed against the SRS before implementation begins.
- Future enhancement: a graphical front end using SFML, replacing RenderModule and MenuModule's console implementation while leaving GameCore, AdminModule, and PersistenceModule unchanged, consistent with the layered architecture chosen in Section 3.5.
- Future enhancement: configurable key bindings, requested informally during review but not yet captured as a requirement.
- Open question: exact file format and location for the local high score file, to be finalized with the team before PersistenceModule implementation.

5. Appendices

5.1 Glossary:

Snake, Food, Game Board, Game Tick, Score, Game Over, Difficulty, HUD, FR, NFR, UML, RTM, ADR, RBAC, STRIDE.

5.2 References:

- Software Requirements Specification (SRS), Snake Game, Team 11, Version 1.0.
- IEEE 42010, Systems and software engineering, Architecture description.
- OWASP guidance on input validation and secure handling of local data.

5.3 Tools:

- PlantUML and draw.io for UML diagrams (component and sequence).
- Makefile and g++ for build and compilation.
- Git for version control.